

Backpropagation: notes

Thomas Serre · Professor of Cognitive & Psychological Sciences · Carney Institute for Brain Science, Brown University · Thursday, October 8 · Fall 2026

You are not expected to derive backpropagation in this course. If you are in computer science, you should be able to; everyone should be able to run it on a small network by hand and read the equations below.

These notes collect the equations of the lecture in one place, with the lecture's worked example. The symbols are the ones on the slides.

Notation

A network has layers $l = 0, 1, \dots, L$. Layer 0 is the input and layer L the output. Layer l has D_l units, its width, and the input has $D_0 = D$ entries. Vectors are bold and are columns. A layer's weights are stored one row per unit, so a layer needs no transpose.

Symbol	Name	Shape
$\mathbf{x} = \mathbf{a}^{[0]}$	input	D_0
$\mathbf{W}^{[l]}$	weight matrix of layer l (row j : the weights into unit j)	$D_l \times D_{l-1}$
$w_{jk}^{[l]}$	weight from unit k of layer $l-1$ to unit j of layer l	scalar
$\mathbf{b}^{[l]}$	bias of layer l , one per unit	D_l
$\mathbf{z}^{[l]}$	weighted sum (pre-activation) of layer l	D_l
g, g'	activation function and its slope, applied to each entry	–
$\mathbf{a}^{[l]}$	activation of layer l	D_l
$\hat{\mathbf{y}} = \mathbf{a}^{[L]}$	prediction (output of the last layer)	D_L
$\mathbf{y}$	target	D_L
ℓ	loss on one example	scalar
$\mathcal{L}$	loss averaged over the examples used for one step	scalar
$\delta^{[l]}$	error of layer l , $\partial\mathcal{L}/\partial\mathbf{z}^{[l]}$	D_l
η	learning rate	scalar
$\odot$	entry-by-entry product of two vectors of the same length	–

A gradient $\partial\mathcal{L}/\partial\mathbf{W}^{[l]}$ has the shape of $\mathbf{W}^{[l]}$: one entry per weight, each saying how much the loss would change if that weight were nudged. For a single example, $\mathcal{L} = \ell$. With several examples, $\mathcal{L}$ is their average, and so is its gradient.

Forward pass

Set $\mathbf{a}^{[0]} = \mathbf{x}$. Then, for $l = 1, \dots, L$,

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}, \quad \mathbf{a}^{[l]} = g(\mathbf{z}^{[l]}),$$

and the prediction is $\hat{\mathbf{y}} = \mathbf{a}^{[L]}$. Unit j of layer l computes one model neuron, $z_j^{[l]} = \sum_k w_{jk}^{[l]} a_k^{[l-1]} + b_j^{[l]}$, and g acts on each entry of $\mathbf{z}^{[l]}$ separately. The hidden layers here use $\text{ReLU}(z) = \max(0, z)$, whose slope $g'(z)$ is 1 when $z > 0$ and 0 otherwise. The output layer uses a sigmoid σ for one yes/no output, or a softmax for one output per class. The loss is the cross-entropy, which for one sigmoid output is

$$\ell = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})],$$

and $-\log \hat{y}$ when $y = 1$. Keep every $\mathbf{z}^{[l]}$ and $\mathbf{a}^{[l]}$: the backward pass uses them.

Backward pass

Output error. With a sigmoid or softmax output and the cross-entropy loss, the error of the output layer is prediction minus target,

$$\delta^{[L]} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}} = \hat{\mathbf{y}} - \mathbf{y}.$$

For one sigmoid output, $\partial \ell / \partial \hat{y} = (\hat{y} - y) / (\hat{y}(1 - \hat{y}))$ and $\sigma'(z) = \hat{y}(1 - \hat{y})$, and their product is $\hat{y} - y$. This is the gradient of logistic regression from earlier in the lecture.

Error sent backward. For $l = L - 1, \dots, 1$,

$$\delta^{[l]} = (\mathbf{W}^{[l+1]^\top} \delta^{[l+1]}) \odot g'(\mathbf{z}^{[l]}).$$

The transpose sends each error back along the forward connections, weighted by them. Unit j of layer l collects $\sum_i w_{ij}^{[l+1]} \delta_i^{[l+1]}$ from the units it feeds and multiplies it by its own slope. A ReLU unit that was off ($z \leq 0$) has slope 0 and gets no error.

Gradients. For every layer,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \delta^{[l]} \mathbf{a}^{[l-1]^\top}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \delta^{[l]}.$$

Entry by entry, $\partial \mathcal{L} / \partial w_{jk}^{[l]} = \delta_j^{[l]} a_k^{[l-1]}$: the error of the unit the weight feeds times the activity of the unit it comes from. A weight whose input was 0 gets a gradient of 0 on this example.

Update. Every weight and bias takes a step against its gradient,

$$\mathbf{W}^{[l]} \leftarrow \mathbf{W}^{[l]} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}}, \quad \mathbf{b}^{[l]} \leftarrow \mathbf{b}^{[l]} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}}.$$

A negative gradient makes the weight grow.

Why it works. The loss depends on a weight only through a chain of quantities, for example $w_{11}^{[1]} \rightarrow z_1^{[1]} \rightarrow a_1^{[1]} \rightarrow z^{[2]} \rightarrow \mathcal{L}$. The chain rule says that the derivative along such a chain is the product of the local slopes, each evaluated at the values of the forward pass. Backpropagation multiplies these slopes starting from the loss and moving toward the input. The product from the loss back to $z^{[l]}$ is $\delta^{[l]}$. It is shared by every weight into layer l and by every earlier layer, so it is computed once and reused, and each weight adds only one multiplication by its input activity. One backward pass therefore costs about as much as one forward pass. Nudging each weight in turn and rerunning the network would cost one forward pass per weight.

Worked example: a 2-2-1 network

The lecture's example is the smallest network that can compute XOR: two inputs, two ReLU hidden units, one sigmoid output, and the cross-entropy loss. One training example, $x = (1, 0)$, whose XOR is $y = 1$. Biases start at 0. The starting weights are

$$\mathbf{W}^{[1]} = \begin{pmatrix} 0.50 & -0.40 \\ 0.30 & 0.80 \end{pmatrix}, \quad \mathbf{W}^{[2]} = \begin{pmatrix} 0.70 & -0.60 \end{pmatrix}.$$

Row 1 of $\mathbf{W}^{[1]}$ holds the weights into hidden unit 1: 0.50 from x_1 and -0.40 from x_2 . Row 2 holds the weights into hidden unit 2: 0.30 from x_1 and 0.80 from x_2 . Numbers are rounded to two decimals. Each step uses the unrounded values of the step before.

Step 1: forward pass

$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]} \mathbf{x} = \begin{pmatrix} 0.50 \cdot 1 - 0.40 \cdot 0 \\ 0.30 \cdot 1 + 0.80 \cdot 0 \end{pmatrix} = \begin{pmatrix} 0.50 \\ 0.30 \end{pmatrix}, \quad \mathbf{a}^{[1]} = \text{ReLU}(\mathbf{z}^{[1]}) = \begin{pmatrix} 0.50 \\ 0.30 \end{pmatrix}.$$

Both hidden units are on, so the ReLU passes their sums unchanged. Then

$$z^{[2]} = 0.70 \cdot 0.50 - 0.60 \cdot 0.30 = 0.17, \quad \hat{y} = \sigma(0.17) = 0.54, \quad \mathcal{L} = -\log 0.542 = 0.61.$$

Step 2: error at the output

$\delta^{[2]} = \hat{y} - y = 0.54 - 1 = -0.46$. It is negative: $z^{[2]}$ should have been higher.

Step 3: error sent backward

Both hidden units were on, so $g'(\mathbf{z}^{[1]}) = (1, 1)$ and

$$\boldsymbol{\delta}^{[1]} = (\mathbf{W}^{[2]\top} \delta^{[2]}) \odot g'(\mathbf{z}^{[1]}) = \begin{pmatrix} 0.70 \cdot (-0.458) \\ -0.60 \cdot (-0.458) \end{pmatrix} \odot \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} -0.32 \\ 0.27 \end{pmatrix}.$$

Each hidden unit gets its own error, with its own sign. Unit 1 excites the output, so it should have been more active. Unit 2 inhibits the output, so it should have been less active.

Step 4: gradients

Error times activity:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[2]}} = \delta^{[2]} \mathbf{a}^{[1]\top} = -0.46 (0.50, 0.30) = (-0.23, -0.14), \quad \frac{\partial \mathcal{L}}{\partial b^{[2]}} = -0.46,$$
$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[1]}} = \delta^{[1]} \mathbf{x}^\top = \begin{pmatrix} -0.32 \\ 0.27 \end{pmatrix} (1, 0) = \begin{pmatrix} -0.32 & 0 \\ 0.27 & 0 \end{pmatrix}, \quad \frac{\partial \mathcal{L}}{\partial b^{[1]}} = \begin{pmatrix} -0.32 \\ 0.27 \end{pmatrix}.$$

The two weights from x_2 get a gradient of 0, because $x_2 = 0$ on this example.

Step 5: one update

With $\eta = 1$, subtract each gradient from its weight or bias:

$$\mathbf{W}^{[1]} = \begin{pmatrix} 0.82 & -0.40 \\ 0.03 & 0.80 \end{pmatrix}, \quad \mathbf{b}^{[1]} = \begin{pmatrix} 0.32 \\ -0.27 \end{pmatrix}, \quad \mathbf{W}^{[2]} = \begin{pmatrix} 0.93 & -0.46 \end{pmatrix}, \quad b^{[2]} = 0.46.$$

Run the forward pass again: $\mathbf{z}^{[1]} = (1.14, -0.25)$, so $\mathbf{a}^{[1]} = (1.14, 0)$, $z^{[2]} = 1.52$, and $\hat{y}$ rises from 0.54 to 0.82. The loss falls from 0.61 to 0.20. Hidden unit 2 is now off for this input, so on the next step it would get no error. This was one step on one example. Training repeats it over all the examples, many times.

Autograd

In practice nobody writes the backward pass. PyTorch records every operation of the forward pass in a computational graph. `loss.backward()` sweeps the graph in reverse, applies the chain rule at each operation, and stores in each parameter's `.grad` exactly the gradients of Step 4. `optimizer.step()` then performs Step 5. For the network above:

```
import torch
import torch.nn.functional as F
W1 = torch.tensor([[0.50, -0.40], [0.30, 0.80]], requires_grad=True)
W2 = torch.tensor([[0.70, -0.60]], requires_grad=True)
x, y = torch.tensor([1.0, 0.0]), torch.tensor([1.0])
z2 = W2 @ torch.relu(W1 @ x)          # forward pass (biases 0)
loss = F.binary_cross_entropy_with_logits(z2, y)
loss.backward()                       # fills W1.grad and W2.grad: Step 4
```

Backpropagation is reverse-mode automatic differentiation applied to a network: the chain rule, applied operation by operation to the numbers computed on the forward pass.